

アーキテクチャと設計判断

目次

本システムの最重要の設計判断は、「判断」と「機械処理」を層で分けたこと。

図 1

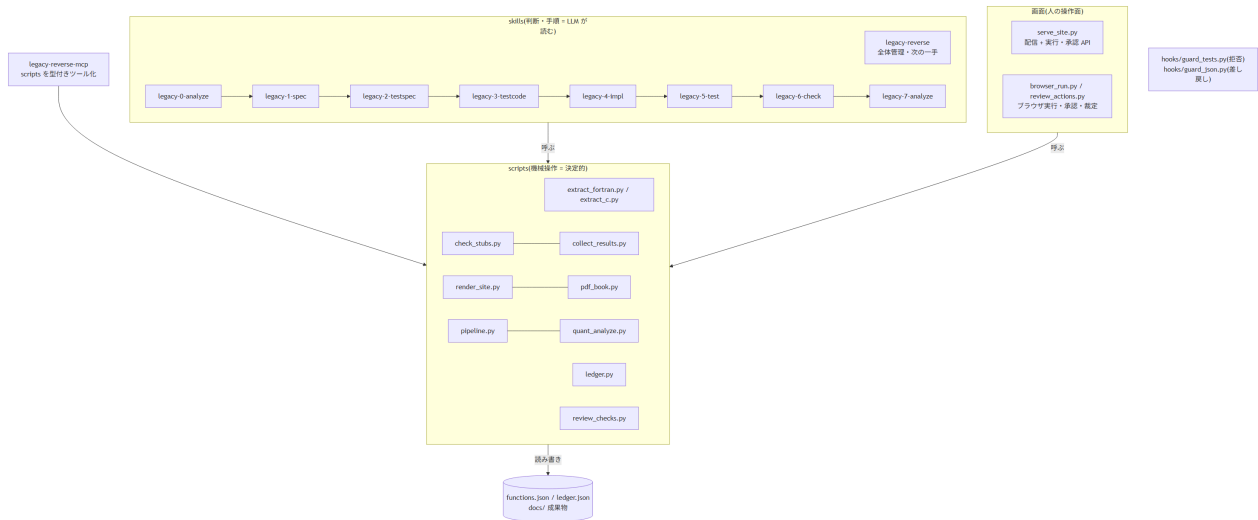


表 1

層	中身	設計判断
skills	「いつ・何を・どの順で」の手順書。実作業の判断(レガシー読解・仕様執筆・トリージ)だけを LLM が行う	LLM は数え上げ・照合・台帳操作をすると誤る(ハルシネーション・省略)。判断以外を持たせない
scripts	単体でも動く決定的処理	列挙・検証・状態管理は再現性が命。LLM を通さない
MCP	scripts を import/subprocess するだけの型付きラップ	二重管理なし。ロジックは常に scripts 側が正
hooks	PreToolUse は呼び出しを拒否、PostToolUse は結果を検証して差し戻す	「プロンプトで禁止」は破られ得る。強制は仕組みで
画面	人の操作(トリガ・承認・裁定)の入口。判断もロジックも持たない	LLM に渡すツールではないので MCP 化しない。実行の実体は scripts 側と共有する

skills から scripts への呼び出しは、MCP 登録済み環境では **MCP ツール優先**、未登録なら直接実行。どちらでも実体は同一(運用設計 — 導入参照)。

実行の入口は 3 つあるが実体は 1 本。チャットの skill・CLI の無人バッチ・ブラウザのボタンは、いずれも `pipeline.run_one(headless Claude 1 回 + ファイル状態での 契約検証)` に合流し、同じ実行枠ロックを取り合う。入口を増やしても実行系統は分岐しない (画面設計)。

フェーズ・スキルの構成

スキルは「全体管理 1 + フェーズ別 8」に分割されている。フェーズごとに分けるのは、(1) 情報遮断を「そのフェーズの skill には読んではいけない入力」が書かれていない」形で構造化するため、(2) 人がトリガを引く単位と一致させるため。

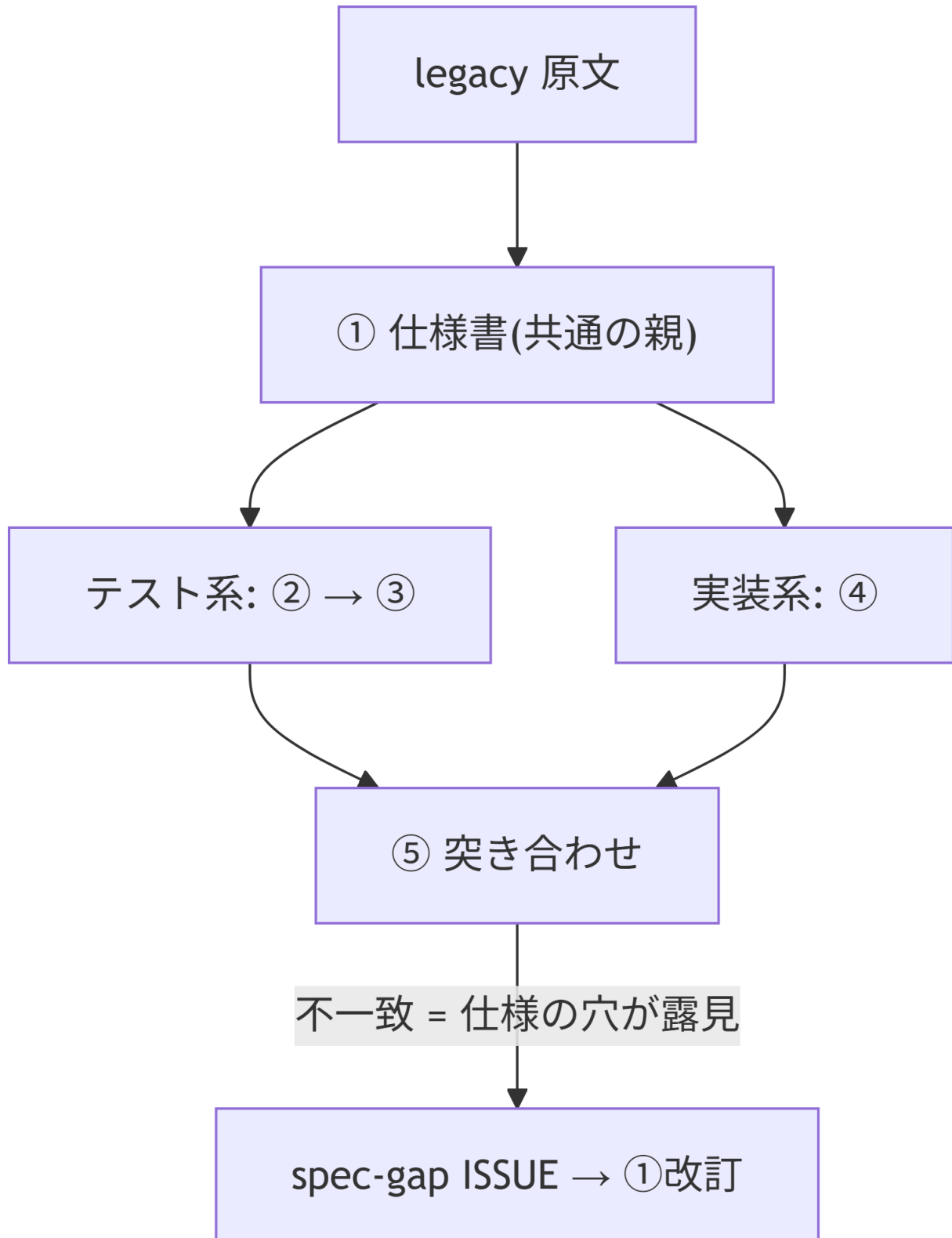
表 2

skill	フェーズ	責務
legacy-reverse	—	セットアップ確認・進捗要約・次アクション提案(状態はスクリプトに聞く)
legacy-0-analyze	①	解析 → functions.json・骨子・WBS・規約(Fortran の列挙は機械抽出。LLM は説明の充填とレポート確認のみ)
legacy-1-spec	①	仕様書執筆。 レガシーを読める唯一の役割 (spec-gap 改訂も担当)。バッチモードあり
legacy-2-testspec	②	テスト仕様。①のみ入力。人の承認ゲート
legacy-3-testcode	③	テストコード。②のみ入力。完了時 freeze でハッシュ固定
legacy-4-impl	④	実装。①のみ入力。スタブ禁止
legacy-5-test	⑤	実行・結果収集・トリアージ・④⑤ループ管理
legacy-6-check	⑥	完了検証と最終レンダリング
legacy-7-analyze	⑦	分析・改善(挙動保存。テスト全 pass 維持が絶対条件)

クリーンルーム分業の意図

②と④が互いを見ない・レガシーを見ないことが品質保証の核。

図 2



- ③が①やレガシーを見られると「実装に合わせたテスト」が書ける(=検証にならない)
- ④がレガシーを見られると「写経」になり、仕様書に書かれていない暗黙挙動が残る

- どちらも遮断すると、仕様書の穴は⑤で機械的に(テスト失敗として)露見する
- ④が詰まったとき自力でレガシーを見に行くのではなく、spec-gap ISSUE を起票 → ①改訂エージェント(レガシーを読める唯一の役割)が仕様を更新 → ハッシュ伝搬 → ④再開

遮断表の正は references/workflow.md。パイプライン仕様に転載。

表 3

ファイル	責務	設計メモ
extract_fortran.py	Fortran 静的解析 → functions.json 生成/マージ	固定・自由形式、継続行、COMMON/USE/intent、call+関数参照の呼出推定。 再実行 = マージ (func_id 不変・手修正保持) が再開性の要
extract_c.py	C/C++ 静的解析 → 同じ functions.json にマージ	Fortran↔C の呼び出しをアンダースコア規約込みで突合。 抽出の実行順に依存しない (後から走った側が未解決名を解決する)
ledger.py	台帳・状態判定・WBS・骨子・検証・⑥check	状態判定 <code>status_of()</code> が唯一の判定ロジック(WBS / next / check が共有)。200 関数超で WBS を自動分割。 正データが壊れていたら行番号つきで停止する (黙って空扱いにしない)
review_checks.py	①②の機械レビュー・一斉レビュー表の生成	LLM 不使用。書式はテンプレート(assets/templates)に依存。表は「承認できる行」が上に来る順で出す
render_site.py	docs/ → HTML サイト・ウィジェットの焼き込み	Quarto が Mermaid を .qmd でしか描けないため影コピー(_sitework)方式。 差分レンダが既定 (2000 関数で数十秒)。承認・実行ボタンはここで各ページに埋め込まれる = UI は再レンダ時点のスナップショット
pipeline.py	無人バッチドライバ・実行ループの実体	1 関数 = 1 headless Claude プロセス (毎回まっさらなコンテキスト = トークン上限に依存しない)。完了は LLM の申告でなくファイル状態で契約検証。中断安全・コスト集計・連続失敗停止。 <code>run_one</code> は 3 つの入口が共有する
browser_run.py	ブラウザからの単発実行・連続実行・残タスク走査	<code>_decide_kind</code> が「次に着手すべき 1 工程」を決める唯一の判定。連続実行は 1 件ごとに再走査するので、承認が下りた関数を自動で拾う
review_actions.py	①②の承認・修正依頼、⑤の裁定	承認時にサーバ側で機械レビューを再実行してから確定 (クライアント表示を信用しない)。完了後に WBS・一斉レビュー表・サイトを再生成
serve_site.py	ローカル配信 + 実行・承認 API	127.0.0.1 のみ・プロジェクト別固定ポート・ <code>--watch</code> 。書き込み POST は 4 段のガード (接続元・Host/Origin・EXE 判定・ルート限定)。

ファイル	責務	設計メモ
		人の操作の入口なので MCP 化しない
collect_results.py	⑤結果収集 → 報告書生成	exit code が制御信号(0 pass / 1 fail / 2 blocked / 3 mismatch)。attempt 上限 3 で自動 block
check_stubs.py	④のスタブ検出	空実装・NotImplementedError・TODO/FIXME
quant_analyze.py	⑦の定量計測(LLM 不使用)	cProfile + radon / ruff / bandit / pip-audit を機械実行し quant.json・quant.md に残す。計測せずに提案しないための 1 段目
pdf_book.py	種別ごとの合本 PDF	quarto-typst-pdf skill に委譲
build_viewer.py	配布用単体実行ファイル	サイト+配信サーバを PyInstaller で 1 ファイル化。受け手に Python / Quarto 不要。閲覧専用(書き込み API は 403)
hooks/ guard_tests.py	④⑤中の tests/ 編集拒否 (PreToolUse)	state.json(phase-start / end)を見て判定
hooks/ guard_json.py	編集直後の JSON 構文検証 (PostToolUse)	壊れていたら exit 2 でエージェント自身に差し戻す。正データの破損でバッチが止まるのを源流で断つ

スケール設計(2000 関数級)

会話コンテキストに全関数リストを載せない、が一貫した方針。

- **WBS の自動 2 段構成:** 200 関数以下は 1 ページ。超えるとトップが「要対応・次の一手・ファイル別進捗」のダッシュボードになり、明細は docs/wbs/ に分割
- **軽量 API:** `ledger status --summary`(数行の JSON 要約)と `ledger next --all` (着手可能リスト)。再開・状況把握はこの 2 つで足りる
- **再開手順は常に同じ:** summary → next → review all。④の再抽出もマージなので安全
- 呼び出し推定は全 function 名を 1 本の交替正規表現に畳んで走査($O(n^2)$ 回避)
- ①の全件実行は会話内ループ禁止(コンテキスト上限・コンパクション劣化)。pipeline.py で 1 関数 1 プロセスに分解する

拡張ポイント

表 4

拡張	方法
新レガシー言語(C# 等)	extract_fortran.py / extract_c.py と同じ出力契約(functions.json スキーマ + extract-report)で抽出器を追加し、MCP ツールに登録。既存の抽出結果とはマージされ、実行順は問わない
新フェーズの自動運転化	①～⑤は実装済み(browser_run.KINDS に「プロンプト・検証関数・リトライ数」を1行足す形)。⑥は LLM 不使用の同期実行、⑦は計測 → 提案の2段。 新しい工程を足すときも骨格は同じ : 対象選定 → headless 実行 → ファイル状態で契約検証 → ログ
意味レベルのレビュー強化	別コンテキストのレビューエージェント(①と legacy だけを読む)をドライバ段に挿入
⑦の改善適用ループ	現状は「計測 → 提案」まで。施策票の承認 → 適用 → 再計測 → 達成判定のイタレーションは形が大きく異なるため別設計
新言語ターゲット	現状 Python+pytest。将来 JS / Rust 拡張を想定

関連: 画面設計 — 実行・承認の入口としての画面層の詳細